

Link list

Data Structure(CSL202)

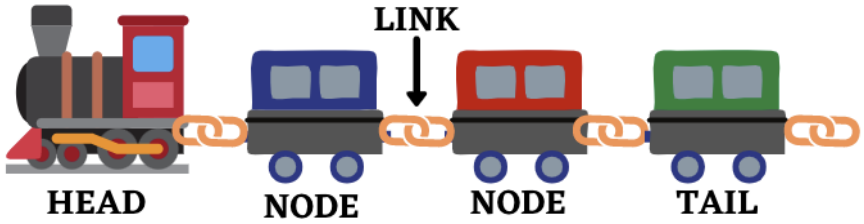
By-
Dr. Sk Subidh Ali (subidh@iitbhilai.ac.in)

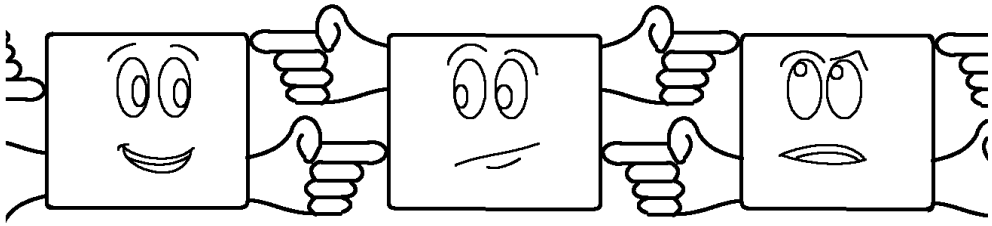


Linked List

Linked List

- Linear collection of data elements, called nodes.
- Linearly connected by links.







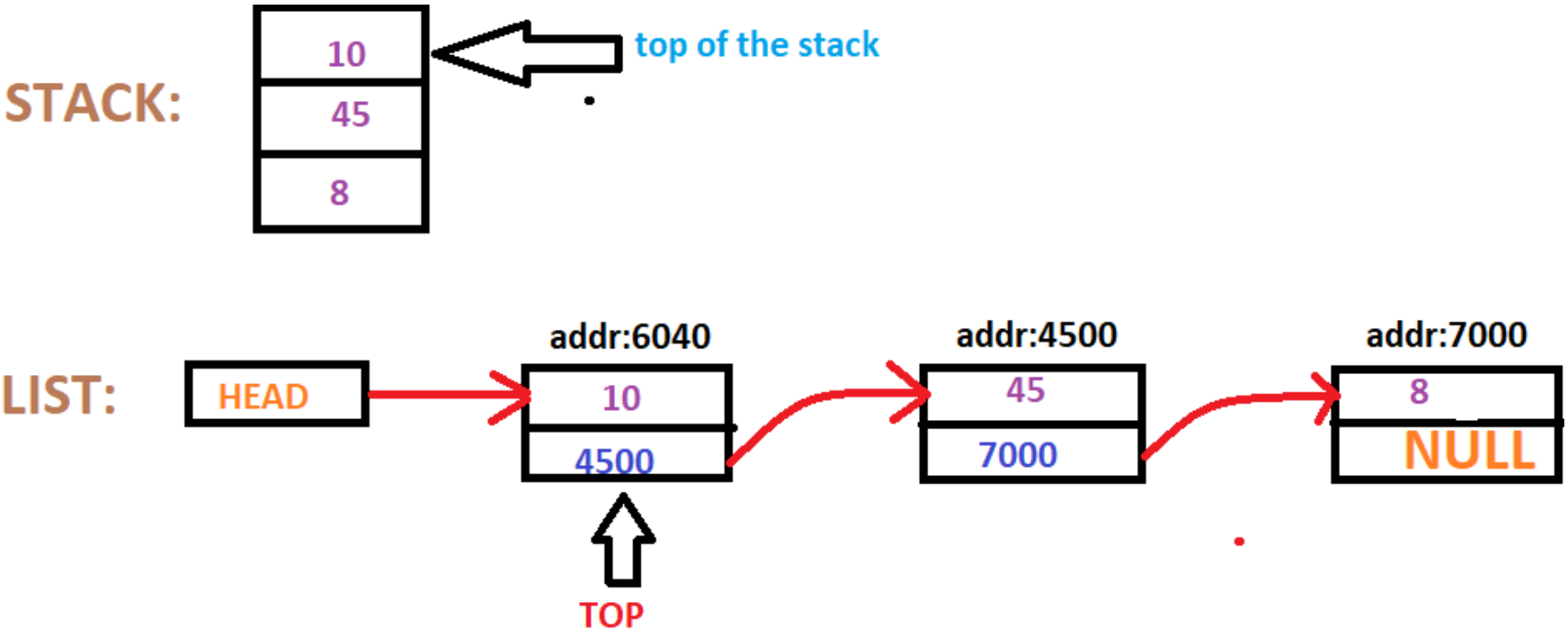


Applications of Linked List

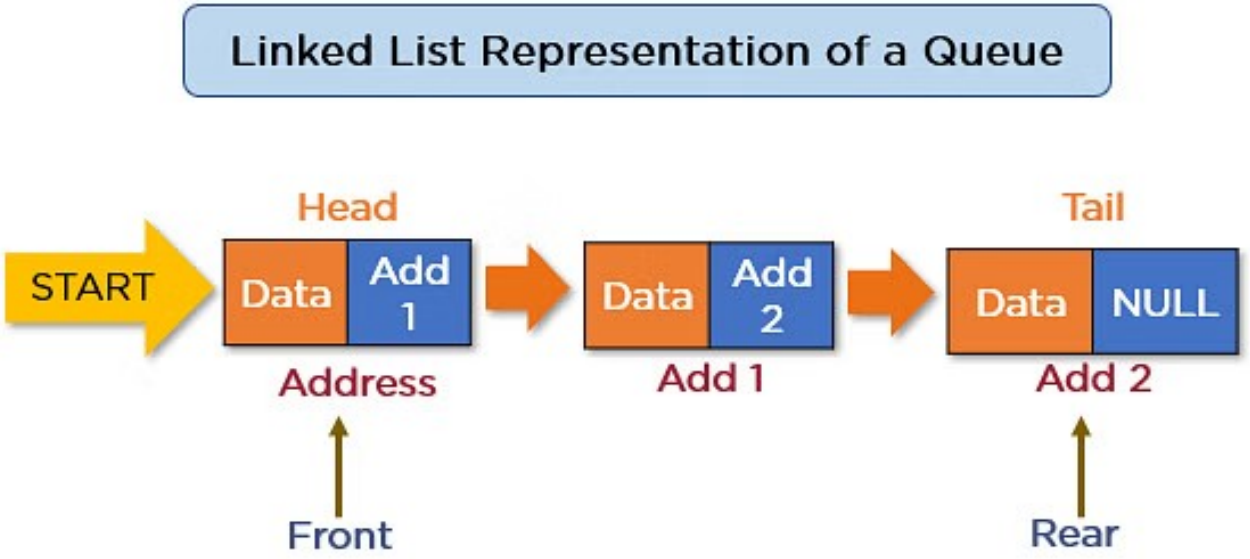
Dynamic memory allocation

- A linked list of free blocks is used.

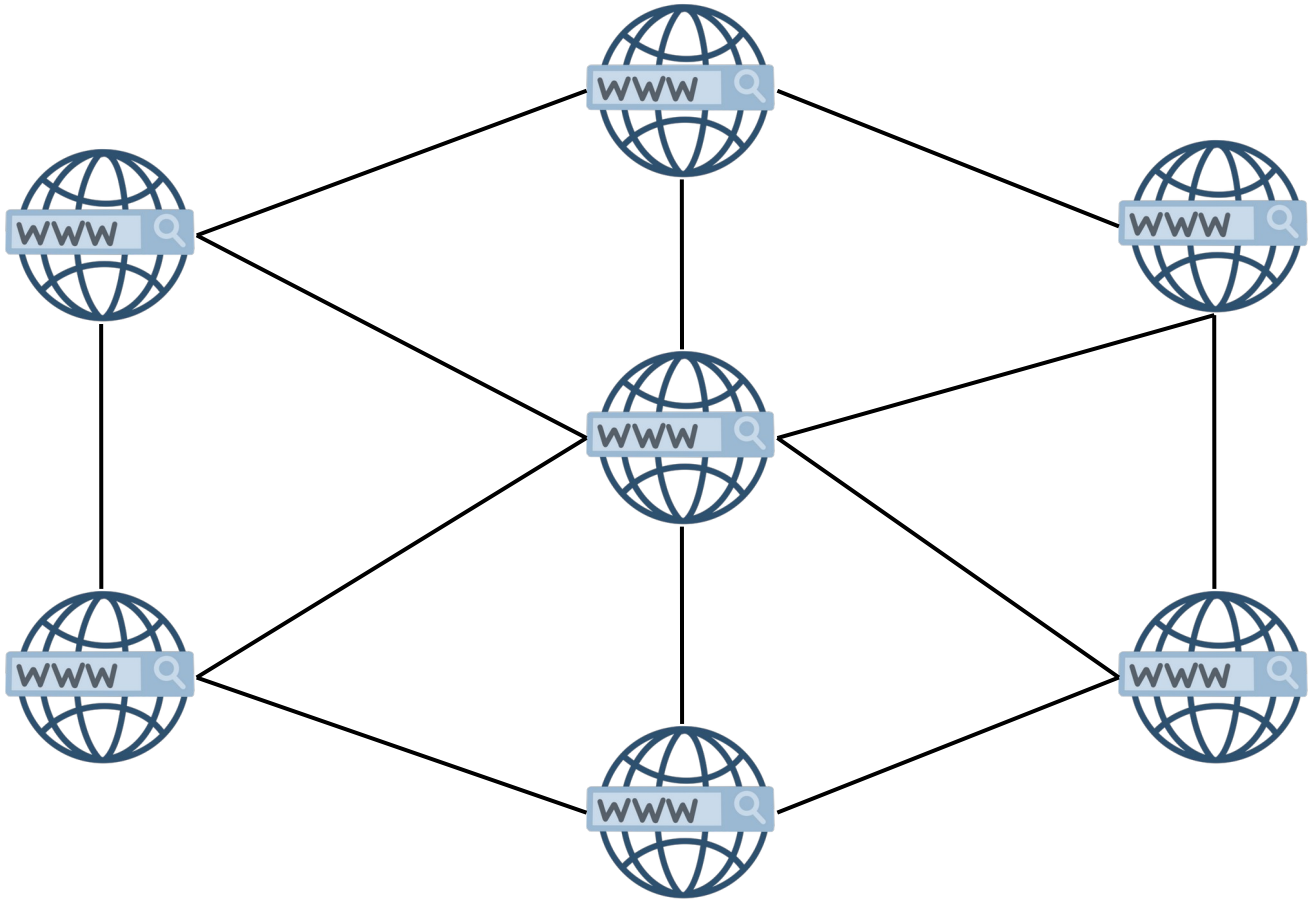
Implementation of stacks



Implementation of queues

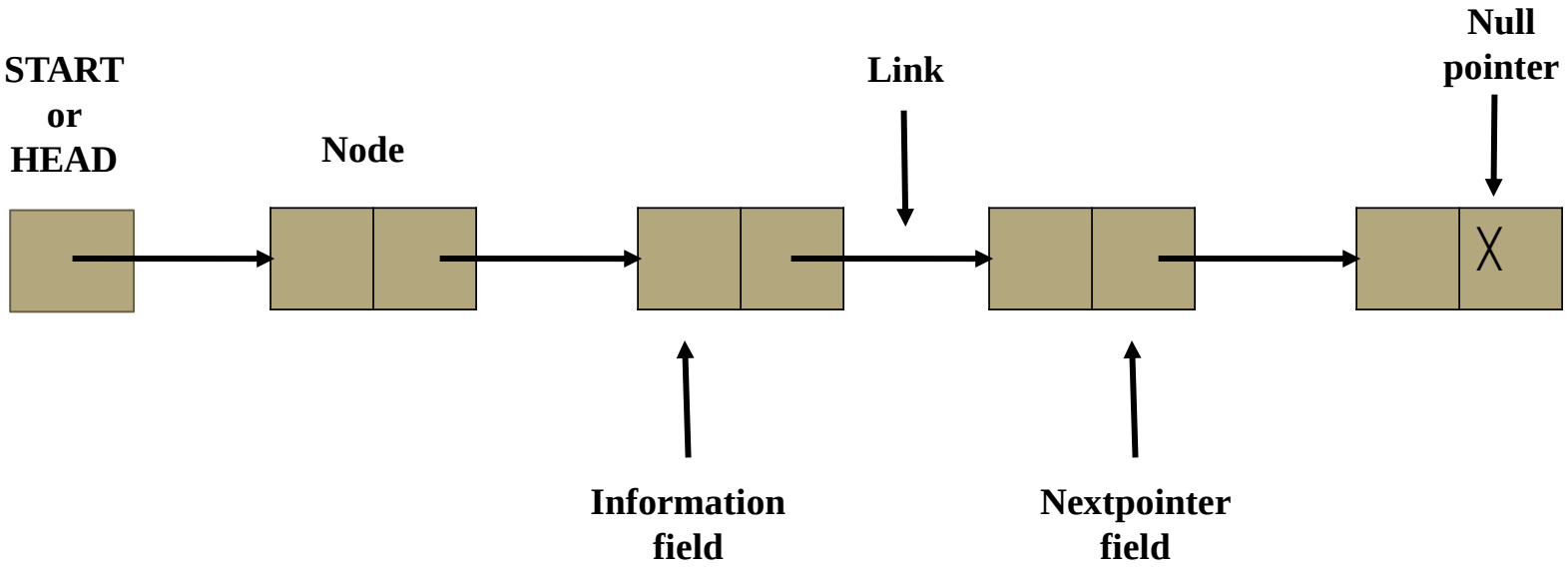


Previous and next page in a web browse



Singly Linked List

- Each node is divided into two parts:
  - First part contains the information.
  - Second part contains the address of the next node in the list.

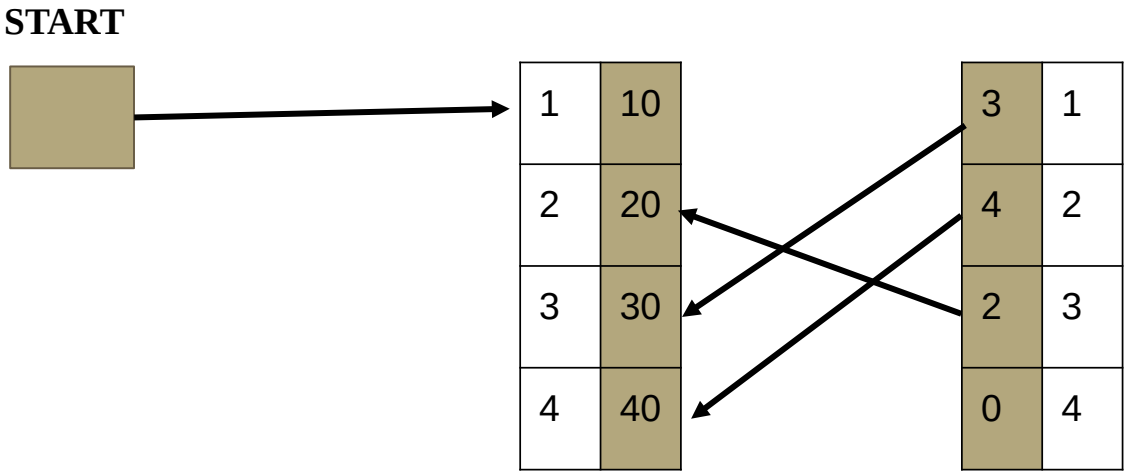




Representation of Linked List in Memory

Representation of Linked List in Memory

- Requires to linear arrays:
  - **INFO[]:** Stores the data (information part) of each node.
  - **LINK[]:** Stores the next pointer (i.e., index of the next node in the list).





Traversing a Linked List

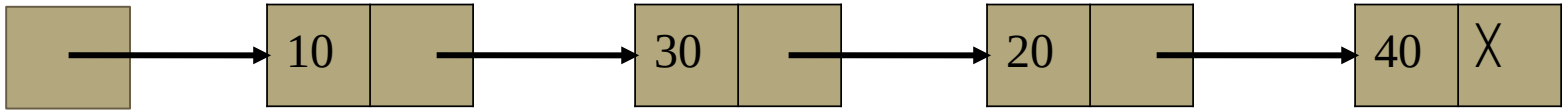
Traversing a Linked List

```
void printList(int INFO[], int LINK[], int START)
{
        int PTR = START;
                while (PTR != NULLPTR)
                {
                printf("%d ", INFO[PTR]);
                PTR = LINK[PTR];  // Move to next node
                }
}
```

Best Case:
- O(1), list empty.

Average Case:
- O(n), half nodes visited.

Worst Case:
- O(n), full list traversed.



10 → 30 → 20 → 40



Searching in a Linked List

Searching in a Linked List

```
void search(int INFO[], int LINK[], int START, int ITEM, int *LOC)
{
        int PTR = START;
        // Traverse while not end of list
                while (PTR != NULLPTR) {
                                if (ITEM == INFO[PTR]) {
                                *LOC = PTR;
                                return;  // Found, exit early
                                } else {
                                PTR = LINK[PTR];  // Move to next node
                                }
                }
        //Item not found
                *LOC = NULLPTR;
}
```

Searching in a Linked List

Time Complexity:

- **Best Case:** O(1) - if ITEM found in first node.
- **Average Case:** O(n) — ITEM found around middle of list.
- **Worst Case:** O(n) — ITEM not found or at last node.